SYSTEM HANDOVER

Handover Document — Abeokuta Polo Festival 2026 Web Platform

This document is written for a successor who did not build the system. It is honest about what is strong, what is manual, and what still needs attention.

PREPARED FOR	The Abeokuta Polo Club (incoming system owners)
EDITION	Grand Finale, 18–20 December 2026
LIVE SITE	https://www.abeokutapolofestival2026.org.ng
STATUS AT HANDOVER	live, no test data, no test prices, all fees at official brochure rates

1. What this system is

A public website that sells and administers admission to the festival: guests register for the free Days 1 and 2, buy VIP Table packages for the Grand Finale on Day 3, receive sealed digital passes that unlock on their phone, and are admitted or refused at the turnstiles by a QR scanner. Behind it sits an operations console for the Club's staff, a money ledger, and an audit trail.

The one rule that governs everything: *a guest has paid only when the Accounts has verified the money.* Until then they hold a **Provisional Claim Slip** and the gate refuses them.

2. Where everything lives

Thing	Where
Public website	https://www.abeokutapolofestival2026.org.ng
Operations console	https://www.abeokutapolofestival2026.org.ng/abawole
Turnstile scanner (mobile)	https://www.abeokutapolofestival2026.org.ng/scan
Source code	github.com/scholarsuite25-netizen/abeokutapolofestival2026 (branch `main`)
Database	Supabase project `zfpybgychouxwarrnkgp` (Postgres, row-level security on every table)

Thing	Where
Hosting	Vercel project `abeokuta-polo-festival-2026`
Email	Resend (primary), EmailBump (fallback)
Club bank	WEMA BANK · ABEOKUTA POLO CLUB · 0126769779
Secretariat	+234 818 188 8884 · 0803 681 8916 · abeokutapolo@gmail.com

For the Club, in PDF: `docs/HANDOVER.pdf` (this document) and `docs/CONSOLE_WALKTHROUGH.pdf` (the operating guide for every console screen, written for staff with no technical knowledge). Both are print-ready and can be shared, signed and filed.

Also in the repository: `docs/CONSOLE_WALKTHROUGH.md` and `docs/HANDOVER.md` (the source of the two PDFs), `ARCHITECTURE_PLAN.md` (the technical design), and `scripts/build-docs-pdf.mjs` (regenerates both PDFs from the markdown).

---

3. Who is in charge of what

Role	Login	Owns	May open
Super Admin (2 accounts)	`admin@`, `superadmin@`	The system itself: creating and removing admins, roles, flags, settings	Everything
Club President	`president@`	The event: participants, finance, gates, reports, invitations	All operations
General Secretary	`secretary@`	Guest-facing operations: registrations, invitations, communications, gate decisions	All operations
Financial Secretary / Accounts	`finsec@`	**The money**: verifying bank transfers, the ledger, sponsor commitments, reports	Participants, finance, gates, reports, invitations
Gate Marshal (3 accounts)	`gate@`, `gate1@`, `gate2@`	The gates on the day	**Scanner only** — no console, no finance

Module ownership at handover:

Module	Accountable	Responsible day-to-day
Public website content & prices	President	General Secretary
Registrations & guest passes	General Secretary	Secretariat desk
Bank transfers & the ledger	**Accounts**	Accounts

Module	Accountable	Responsible day-to-day
VIP Tables (incl. which table is sold)	Accounts	Secretariat desk (availability), Accounts (money)
Complimentary VIP invitations	**General Secretary**	Secretariat desk
Turnstile scanning & gate incidents	President	Gate Marshals
Reports & exports	President	Accounts
Admin accounts & permissions	Super Admin	Super Admin
Database changes	Super Admin	Super Admin (always via a migration, never by hand)

4. Day-to-day operating

Full instructions are in `docs/CONSOLE_WALKTHROUGH.md`. The short version:

- **Day 1 & 2** — guests register free online; marshals scan at Gate 2; export the daily attendance log at close.
- **Day 3** — the Accounts clears every pending bank transfer **before** the gate opens; any table still showing *Amount Paid = 0* stays refused; export the finance ledger and the master roster for the archive.
- **Every evening** — Accounts reconciles the day's transfers against the bank statement.

5. The money rules (do not change these without the President's written approval)

1. **Day 3 admission is sold only with a VIP Table.** There is no standalone Day-3 ticket. The ₦50,000 Regular Section seat is sold on the Tables page only.
2. **Official brochure prices only.** ₦5,000,000 Kings · ₦3,000,000 Royal · ₦2,000,000 Queens · ₦500,000 Premium · ₦50,000 Regular Section seat. There is no test price and no switch.
3. **Prices are decided by the server.** The browser cannot influence what is charged — this is enforced in code and by the `price_list` table.
4. **A bank transfer is not a payment until the Accounts verifies it.** Unverified transfers get a Provisional Claim Slip, are marked *Pending Bank Verification*, and are refused at the gate.
5. **A table is sold once.** A unique database constraint prevents the same table from being sold twice, and one transfer cannot pay for two bookings.
6. **Commission** is 10% and is only credited once a booking is verified.

6. The database

Ownership: Supabase Postgres, project `zfpybgychouxwarrnkgp`. Every table has row-level security enabled with no policies and is revoked from `anon` / `authenticated` — **only the server (service-role key) can read or write.** Browser-facing access to data is impossible by design.

Table families

Family	Tables	Purpose
Event configuration	`festival_days`, `gates`, `hospitality_packages`, `price_list`, `affiliate_commission_tiers`	The 3 days, the 3 gates, the 5 packages, price history, commission tiers
Attendees	`registrations`, `credentials`, `attendee_devices`, `attendee_notes`	Who is registered, their sealed pass, the phone it locked to, internal notes
Hospitality	`table_bookings`, `table_seats`, `table_holds`	Table reservations, the seat ledger, temporary holds during checkout
Money	`payment_ledger`, `bank_transfer_receipts`, `refunds`, `affiliate_payouts`, `commitment_payments`, `offline_commitments`, `day3_payment_claims`	One ledger for every naira, receipts, refunds, partner payouts, commitments, single-use card claims
Invitations	`invitations`, `invitation_audit`	Complimentary VIP passes and every action taken on them
Gate operations	`check_in_events`, `gate_incidents`, `scanner_devices`, `denied_admissions`	Who came through, every manual override, the scanner fleet, refused attempts
Communications	`notification_log`, `reminder_log`, `message_templates`	Proof of every email sent, reminder history, editable copy
Governance	`feature_flags`, `feature_flag_history`, `site_settings`, `announcements`	Administrator-only switches (never guest-facing), settings, banners
Trust & safety	`fraud_alerts`, `security_events`, `report_exports`, `help_requests`, `audit_logs`	Suspicious activity, security events, who exported guest data, support requests
Identity	`profiles`, `roles`, `permissions`, `role_permissions`, `user_role_assignments`, `audit_logs`	Staff accounts and exactly what each may do

Migrations

SQL lives in `supabase/migrations/` (0011 → 0019) and a consolidated `supabase/APPLY_ALL_MIGRATIONS.sql`.

- **0011 – 0017** — applied.
- **0018 – 0019** — **applied and verified** (booking integrity + the operational schema: 30 tables, row-level security on, `anon` and `authenticated` revoked). Re-applying them is harmless, but there is nothing outstanding.

Rule for the future: never edit the live database by hand. Every schema change is a numbered migration file, committed to the repository, and applied in order. A change made only in the database will be silently lost at the next deploy.

7. Automation

What	When	Notes
Weekly guest reminders	Mondays 08:00	Vercel cron
Daily guest reminders	06:00 on 18, 19, 20 Dec	Vercel cron
Database backup	**Nightly 02:00**	Vercel cron → `/api/ops/backup`; writes a full JSON export to the private `club-backups` storage bucket, keeping 14 days
Pass issuance & verification	Instant	On registration / booking
Gate admission decisions	Instant	At the scanner
CSV report generation	On demand	Reports & Export
Bank reconciliation	❌ **Manual**	The Accounts verifies each transfer by hand — deliberate, because only a human can read a bank statement
Receipts & invoices	❌ **Manual**	Not yet generated automatically
Affiliate payout statements	❌ **Manual**	Commission accrues correctly; statements are not yet generated
Email delivery log	✅ Automatic	Every send recorded in `notification_log`

8. Security posture

How staff sign in (Username & Password only — no codes or authenticator apps)

- **Username & password only.** The staff addresses (`@abeokutapolo2026.ng`) were autogenerated as login usernames and cannot receive email codes. The sign-in page (`/sign-in`) goes straight to **Username + Password** with a Show/Hide toggle.
- **One-click Super Admin recovery ("Approve").** If any administrator or gate marshal has a password or sign-in problem, the Super Admin opens **Admin Hierarchy** (`/abawole/admins`) and clicks **Approve** next to their name. That immediately reactivates the account, restores their role if needed, and displays a fresh password to hand over in person.
- **Passwords are private.** Staff passwords are 20-character random strings, never stored in the public repository and never displayed in the console. The Super Admin can download the official credentials sheet (PDF or `.txt`) directly from `/abawole/admins`.
- **Two-factor authentication is deliberately OFF.** Nothing prompts anyone for a code or authenticator app, and no account is blocked.

Protections that are in place

- **Passes are cryptographically sealed** (HMAC-SHA256) and verified on the server; a forged or edited pass is rejected at the gate.
- **Payment references are single-use** — a replayed reference fails on a database constraint.
- **Passes are device-locked** — they lock to the first device that opens them and cannot be forwarded or resold. A laptop is treated exactly like a phone: the laptop claims the pass, and the guest may hand it over to their phone **once** (audited as `pass.device_activated_on_mobile`), after which every other device is refused.
- **Default-deny permissions** — every console action is checked against a role, and a missing permission means "no".
- **Row-level security everywhere** — the browser cannot read guest data even if it is compromised.
- **Rate limiting is durable** — stored in the database, so it survives redeploys and applies across every server instance. The credential oracle allows 30 attempts a minute; the public registration and booking forms are throttled per caller.
- **Fail-closed defaults** — an unspecified status is an *unpaid* booking, never a confirmed one.
- **Audit trails** — every state change, every email, every data export, every manual gate override is recorded with who did it and when.
- **Transport & headers** — HSTS (with preload), a Content-Security-Policy, `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`, a restrictive `Permissions-Policy`, and `no-store` on exports.
- **Passwords are never in the repository or in the console** — they are rotated through the Supabase admin API and issued to staff out of band.

Known gaps — please close these

1. **Text-message (SMS) sign-in is implemented but not yet switched on.** It needs one dashboard action the engineering team cannot perform without dashboard access: Supabase → Authentication → Sign In / Providers → **Phone**, plus the club's own Twilio account credentials and each staff member's real number attached to their account. The button on the sign-in page stays hidden until then, so nothing is broken in the meantime.
2. **Multi-factor authentication is available but switched off.** Each console account can enrol a second factor at `/mfa-setup` (phone app, laptop, or fingerprint/USB key); a Super Admin can turn on the `mfa_required_for_console` flag and clear a member's factor from Admin Hierarchy if they lose their device. The club has decided not to use it, so nothing is enforced today.
3. **Rotate the admin passwords at handover** — they were issued out of band (see the credentials file) and must be changed by each person.
4. **Old sessions stay valid until they expire.** After a password rotation, sign out other sessions (Supabase → Auth → Users → "Sign out all sessions") so nobody stays signed in with a pre-rotation session.
5. **Dependency advisories** exist in the tree (`npm audit`): all are in build-time/CLI tooling, and the runtime ones (`zod`) are already on a patched version. Re-run `npm audit` before each release.
6. **No automated security scanning** in CI.

---

9. Backup & recovery ⚠️ *action required*

There are now two independent layers of protection:

1. **Nightly export (implemented).** A Vercel cron runs `/api/ops/backup` every night at 02:00. It exports every public table to a single JSON file and stores it in a **private** Supabase Storage bucket (`club-backups`), keeping the last 14 days. Each run is recorded in the audit log as `system.database_backup`.
   - Run one by hand: `curl -X POST https://www.abeokutapolofestival2026.org.ng/api/ops/backup -H "Authorization: Bearer $CRON_SECRET"`
   - Inspect the backups: `node scripts/restore-from-backup.mjs` (dry run — prints what it would restore)
   - Restore: `node scripts/restore-from-backup.mjs --confirm` (and `--file <name>.json` for a specific night)
2. **Provider-side recovery.** Supabase provides automated daily backups and point-in-time recovery *on paid plans* — **confirm this is active on your project today**, because it is the second layer.

Still to do by the Club:

1. **Rehearse a restore once a month** into a scratch project and check the guest count. A backup that has never been restored is not a backup.
2. Write down who holds the credentials for the Supabase account, the Vercel account and the GitHub repository — in a sealed envelope or a password manager the Club controls, not in the repository.

If the database is lost: restore the latest backup with `scripts/restore-from-backup.mjs --confirm` (or use point-in-time recovery to the last good moment), then re-deploy the site from GitHub — the schema is fully described by the migration files, so a restored database and the code always match.

---

10. Incident runbook

Situation	What to do
Guest says their pass will not open	Check the Verification Queue — the pass may be *Provisional* (unpaid) or bound to another phone. Use **Unlock** for a lost phone.
Guest claims they paid but is refused	Ask for the transfer reference, find it in Offline Finance, and press **Verify Table Transfer** once the money is confirmed.
Two guests were sold the same table	This should now be impossible (unique constraint). If it happens, cancel one booking, issue a refund from `refunds`, and apologise with a courtesy seat.
A gate marshal is locked out	A Super Admin resets the password, or deactivates and recreates the account. Never share a login.
Someone is spamming registrations	Check `fraud_alerts` (category `rate_limited`) — the limiter is already blocking them; add their IP to the block list at the edge.
The site is down	Redeploy: `vercel --prod --yes` from the repository (Vercel keeps the last 5 deployments for rollback). Check Vercel status first — it is occasionally the network, not the app.

Situation	What to do
The database looks wrong	Stop. Do not hand-edit. Write a migration, have a Super Admin review it, apply it, and deploy.
A suspicious payment or pass	Open a row in `fraud_alerts` (the app raises these automatically) and record the outcome.

11. Deliberate choices (so nobody "fixes" them by accident)

- **Day 3 is a table-only event.** The standalone Day-3 ticket was removed by decision; the server refuses it.
- **Paystack does not appear in the tables flow.** ARCHITECTURE_PLAN §11 D-01 keeps card processing inside the Day-3 VVIP flow only. This is a rule with a test that enforces it (`no-payments-guard`).
- **The banking details are printed on the page.** Deliberate: guests transfer by hand and submit the receipt to the Secretariat, which is what makes the paywall meaningful.
- **Unverified = refused.** Do not soften this to make the gate queue shorter.

12. Launch checklist

- ☐ Migrations 0018 and 0019 confirmed applied on Supabase *(done — verified 30 tables, RLS on, `anon` blocked)*
- ☐ Point-in-time recovery / backups confirmed, and a restore rehearsed once
- ☐ Every staff member has changed the password they were given in person
- ☐ Old sessions signed out (Supabase → Auth → Users → "Sign out all sessions")
- ☐ SMS codes switched on — Supabase → Authentication → Sign In / Providers → Phone, plus the club's Twilio account and each person's real number attached to their account *(optional)*
- ☐ Dress rehearsal: one real booking on a phone, on mobile data, at a real gate
- ☐ Load test: 100 simultaneous registrations and 200 scans in 10 minutes
- ☐ Staging environment created with a copy of the data, and the Club has rehearsed there
- ☐ Printed gate roster exported (Master Accreditation Roster) and given to each marshal
- ☐ Bank account details on the printed material confirmed current
- ☐ Every staff member has read `docs/CONSOLE_WALKTHROUGH.pdf`
- ☐ The shared mailbox (`abeokutapolo@gmail.com`) is watched by the Secretariat throughout the event

13. Handover contacts

Need	Who
System access / roles / anything broken	Super Admin

Need	Who
Guest complaints, invitations, passes	General Secretary
Payments, refunds, transfers	Accounts
Gate operation on the day	President
Hosting, database, repository	Super Admin (engineer of record)

Repository: `github.com/scholarsuite25-netizen/abeokutapolofestival2026` — branch `main` is the live code; every change is a commit with a message explaining why.

This document describes the system as at handover. Where reality and this document disagree, reality wins — update the document, not the other way round.